Group Number:
Student Names:
Marks: 4 points

COMP 251: Data Structures & Algorithms
Group Assignment 3: Algorithm Analysis

Prepared by: Omer Waqar (Ph.D., P.Eng., SMIEEE)

********** INSTRUCTIONS **********

- This assignment focuses on **algorithmic reasoning and asymptotic analysis**. Answers will be evaluated primarily on correctness of analysis and clarity of justification.
- For **Questions 1** and **3**, provide explanation of how the final time complexity is derived.
- In **Question 2**, explicitly explain how your solution satisfies $O(n)$ time and $O(1)$ additional space constraints.
- All submitted work must be your own (or your group's) original work. Any form of plagiarism or unauthorized collaboration will be handled according to university policies.

Question 1: Big-Oh Characterization

Provide a Big-Oh characterization, in terms of n, of the running times of the `example2` and `example3` methods, shown below

```
1  /** Returns the sum of the integers with even index in given array. */
2  public static int example2(int[ ] arr) {
3  int n = arr.length, total = 0;
4  for (int j=0; j < n; j += 2) // note the increment of 2
5      total += arr[j];
6  return total;
7  }
```

```
8  /** Returns the sum of the prefix sums of given array. */
9  public static int example3(int[ ] arr) {
10     int n = arr.length, total = 0;
11     for (int j=0; j < n; j++) // loop from 0 to n-1
12         for (int k=0; k <= j; k++) // loop from 0 to j
13          total += arr[j];
14 return total;
15  }
```

Question 2: Design an algorithm for a given time complexity

An array A contains $n-1$ **unique** integers in the range $[0, n-1]$, that is, there is one number from this range that is not in A. Design a $O(n)$-time algorithm for finding that number. You are only allowed to use $O(1)$ additional space besides the array A itself.

Question 3: Asymptotic analysis of the insertion-sort algorithm

Consider the following Java code for the insertion-sort algorithm that arranges elements of the given array in ascending order.

```
16
17  public static void insertionSort(int[] arr) {
18          int n = arr.length;
19
20          for (int i = 1; i < n; i++) {
21              int key = arr[i];
22              int j = i - 1;
23
24              // Move elements of arr[0..i-1], that are greater than key, to one
                    position ahead
25              // of their current position
26              while (j >= 0 && arr[j] > key) {
27                  arr[j + 1] = arr[j];
28                  j--;
29              }
30
31              // Place the key in its correct position
32              arr[j + 1] = key;
33          }
34      }
```

What are the worst-case and best-case running times of the insertion-sort algorithm? Provide a complete rationale of your answer.